

Assignment 3

Hadeer Ghoneim

Verification Diploma

Q1

1. Testbench code

```
1  import testing_pkg::*;
2
3  module tb();
4
5      Transaction tr = new();
6      byte operand1, operand2, out, expected;
7      opcode_e opcode;
8      logic clk, rst;
9
10     int correct_count = 0;
11     int error_count  = 0;
12
13     // Clock Generation
14     initial begin
15         clk = 0;
16         forever begin
17             #5 clk = ~clk;
18             tr.clk = clk;
19         end
20     end
21
22     // DUT Instantiation
23     alu_seq DUT (
24         .operand1 (operand1),
25         .operand2 (operand2),
26         .clk      (clk),
27         .rst      (rst),
28         .opcode   (opcode),
29         .out      (out)
30     );
31
32     // -----
33     // Tasks
34     // -----
```

```

36 // Reset task
37 task do_reset();
38     rst = 1;
39     @(negedge clk);
40     rst = 0;
41 endtask
42
43 // Calculate expected output
44 task calc_expected();
45     case (opcode)
46         ADD: expected = operand1 + operand2;
47         SUB: expected = operand1 - operand2;
48         MULT: expected = operand1 * operand2;
49         DIV: expected = (operand2 != 0) ? (operand1 / operand2) : 0;
50         default: expected = 0;
51     endcase
52 endtask
53
54 // Compare DUT vs Expected
55 task check_output();
56     if (out === expected) begin
57         correct_count++;
58         $display("PASS: opcode=%0s, op1=%0d, op2=%0d, out=%0d",
59             | | | | | opcode.name(), operand1, operand2, out);
60     end
61     else begin
62         error_count++;
63         $display("FAIL: opcode=%0s, op1=%0d, op2=%0d, expected=%0d, got=%0d",
64             | | | | | opcode.name(), operand1, operand2, expected, out);
65     end
66 endtask

```

```

67
68 task force_operand1_operand2_cross();
69 static byte corners[3] = {-128, 0, 127};
70 int i, j;
71
72 for (i = 0; i < 3; i++) begin
73     for (j = 0; j < 3; j++) begin
74         operand1 = corners[i];
75         operand2 = corners[j];
76         opcode   = ADD;
77         opcode   = SUB;
78         opcode   = MULT;
79         opcode   = DIV; |
80
81         @(negedge clk);
82         calc_expected();
83         check_output();
84     end
85 end
86 endtask

```

```

89 // -----
90 // Stimulus
91 // -----
92 initial begin
93     do_reset();
94
95     force_operand1_operand2_cross();
96
97     repeat (5000) begin
98         assert(tr.randomize()); // generate transaction
99         operand1 = tr.operand1;
100        operand2 = tr.operand2;
101        opcode   = tr.opcode;
102
103        @(negedge clk); // wait for DUT
104        calc_expected();
105        check_output();
106
107        do_reset();
108    end
109
110    $display("=====");
111    $display("Simulation finished!");
112    $display("Correct Count = %0d", correct_count);
113    $display("Error Count   = %0d", error_count);
114    $display("=====");
115
116    $stop();
117 end
118 endmodule

```

2. Package code

```

1 package testing_pkg;
2
3 typedef enum {ADD, SUB, MULT, DIV} opcode_e;
4
5 class Transaction;
6     rand opcode_e opcode;
7     rand byte operand1;
8     rand byte operand2;
9     bit clk;
10
11     covergroup CovCode @(posedge clk);
12
13         operand1_cp: coverpoint operand1 {
14             bins max_neg = {-128};
15             bins zero     = {0};
16             bins max_pos  = {127};
17             bins misc     = default;
18         }
19
20
21         operand2_cp: coverpoint operand2 {
22             bins max_neg = {-128};
23             bins zero     = {0};
24             bins max_pos  = {127};
25             bins misc     = default;
26         }
27
28
29         opcode_cp: coverpoint opcode {
30             bins add_sub    = {ADD, SUB};
31             bins add_mult   = {ADD, MULT};
32             bins mult_sub   = {MULT, SUB};
33             bins add_then_sub = (ADD => SUB);
34             bins add_then_mult = (SUB => MULT);
35             bins mult_then_sub = (MULT => ADD);
36             illegal_bins no_div = {DIV};

```

```

36         illegal_bins no_div = {DIV};
37     }
38
39     opcode_operand1: cross opcode_cp, operand1_cp {
40         option.weight = 5;
41         option.cross_auto_bin_max = 0;
42
43         bins max_pos_add_or_sub =
44             binsof(operand1_cp.max_pos) && binsof(opcode_cp.add_sub);
45
46         bins max_pos_add_or_mult =
47             binsof(operand1_cp.max_pos) && binsof(opcode_cp.add_mult);
48
49         bins max_pos_mult_or_sub =
50             binsof(operand1_cp.max_pos) && binsof(opcode_cp.mult_sub);
51
52         bins max_neg_add_or_sub =
53             binsof(operand1_cp.max_neg) && binsof(opcode_cp.add_sub);
54
55         bins max_neg_add_or_mult =
56             binsof(operand1_cp.max_neg) && binsof(opcode_cp.add_mult);
57
58         bins max_neg_mult_or_sub =
59             binsof(operand1_cp.max_neg) && binsof(opcode_cp.mult_sub);
60     }
61
62
63     opcode_operand2: cross opcode_cp, operand2_cp {
64         option.weight = 5;
65         option.cross_auto_bin_max = 0;
66
67         bins max_pos_add_or_sub =
68             binsof(operand2_cp.max_pos) && binsof(opcode_cp.add_sub);

```

```

63     opcode_operand2: cross opcode_cp, operand2_cp {
64         option.weight = 5;
65         option.cross_auto_bin_max = 0;
66
67         bins max_pos_add_or_sub =
68             binsof(operand2_cp.max_pos) && binsof(opcode_cp.add_sub);
69
70         bins max_pos_add_or_mult =
71             binsof(operand2_cp.max_pos) && binsof(opcode_cp.add_mult);
72
73         bins max_pos_mult_or_sub =
74             binsof(operand2_cp.max_pos) && binsof(opcode_cp.mult_sub);
75
76         bins max_neg_add_or_sub =
77             binsof(operand2_cp.max_neg) && binsof(opcode_cp.add_sub);
78
79         bins max_neg_add_or_mult =
80             binsof(operand2_cp.max_neg) && binsof(opcode_cp.add_mult);
81
82         bins max_neg_mult_or_sub =
83             binsof(operand2_cp.max_neg) && binsof(opcode_cp.mult_sub);
84     }
85
86
87 endgroup
88
89 function new();
90     CovCode = new();
91 endfunction
92
93 endclass : Transaction
94
95 endpackage : testing_pkg
96

```

3. Design code

```

1  import testing_pkg::*;
2  |
3  module alu_seq(operand1, operand2, clk, rst, opcode, out);
4  input byte operand1, operand2;
5  input clk, rst;
6  input opcode_e opcode;
7  output byte out;
8
9  always @(posedge clk) begin
10     if (rst)
11         out <= 0;
12     else
13         case (opcode)
14             ADD: out <= operand1 + operand2;
15             SUB: out <= operand1 - operand2;
16             MULT: out <= operand1 * operand2;
17             DIV: out <= operand1 / operand2;
18             default: out <= 0;
19         endcase
20     end
21 endmodule

```

4. Report any bugs detected in the design and fix them: no any
5. Snippet to your verification plan document

Label	Design requirement description	Stimulus generation	Functional coverage	Functionality check
ALU_RESET	When reset is asserted, the output out should be zero.	Directed at start	Coverpoint on rst transition + out==0 bin	A checker task compares DUT output against expected=0 after reset.
ALU_ADD	For opcode=ADD, DUT output = operand1 + operand2.	Randomization + Directed corners	Coverpoint on opcode=ADD, cross with operand1_cp &	Checker task calculates expected ADD result and compares with DUT output.
ALU_SUB	For opcode=SUB, DUT output = operand1 - operand2.	Randomization + Directed corners	Coverpoint on opcode=SUB, cross with operand1_cp &	Checker task calculates expected SUB result and compares with DUT output.
ALU_MULT	For opcode=MULT, DUT output = operand1 * operand2.	Randomization	Coverpoint on opcode=MULT, operand1/operand2 directed to 0, 1, max_pos, max_neg	Checker task calculates expected MULT result and compares with DUT output.
ALU_DIV	For opcode=DIV, DUT output = operand1 / operand2 (excluding divide-by-zero).	Randomization + Directed (avoid div/0)	Coverpoint on opcode=DIV, illegal_bins for divide-by-zero	Checker task calculates expected DIV result and compares with DUT output.
ALU_OP1_OP2	Operand1 & Operand2 extreme values (-128, 0, 127) combinations must be exercised.	Directed 3x3 corner test	Cross operand1_cp × operand2_cp, bins for (max_pos, max_neg, zero) combinations	Checker ensures DUT output is consistent with operation for all corner cases.
ALU_OPCODE_SEQ	ALU should cover sequence ADD → SUB (temporal requirement).	Randomization	Coverpoint on opcode_cp.add_then_sub bin	Coverage ensures ADD is followed by SUB at least once; checker compares
ALU_NO_DIV	DIV opcode is marked illegal (by spec).	Constrained random	illegal_bins no_div ensures this bin never gets hit	Simulation must never see DIV when illegal; scoreboard check triggers if violated.

6. Do file

```

1  vlib work
2  vlog alu_seq.sv testing_pkg.sv tb.sv +cover -covercells
3  vsim -voptargs=+acc work.tb -cover
4  add wave *
5  coverage save tb.ucdb -onexit
6  run -all

```

7. Code Coverage report snippets (Questions 1, 2, and 3 only)

code_cvr_alu - Notepad

File Edit Format View Help

Coverage Report by instance with details

=====
=== Instance: /tb/DUT

=== Design Unit: work.alu_seq
=====

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	6	6	0	100.00%

=====Branch Details=====

Branch Coverage for instance /tb/DUT

Line	Item	Count	Source
----	----	-----	-----
File alu_seq.sv			
-----IF Branch-----			
10		10010	Count coming in to IF
10	1	5001	if (rst)
12	1	5009	else

Branch totals: 2 hits of 2 branches = 100.00%

-----CASE Branch-----

13		5009	Count coming in to CASE
14	1	1240	ADD: out <= operand1 + operand2;
15	1	1180	SUB: out <= operand1 - operand2;
16	1	1232	MULT: out <= operand1 * operand2;
17	1	1357	DIV: out <= operand1 / operand2;

Branch totals: 4 hits of 4 branches = 100.00%

code_cvr_alu - Notepad
File Edit Format View Help

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	6	6	0	100.00%

=====Statement Details=====

Statement Coverage for instance /tb/DUT --

Line	Item	Count	Source
----	----	-----	-----
File alu_seq.sv			
3			module alu_seq(operand1, operand2, clk, rst, opcode, out);
4			input byte operand1, operand2;
5			input clk, rst;
6			input opcode_e opcode;
7			output byte out;
8			
9	1	10010	always @(posedge clk) begin
10			if (rst)
11	1	5001	out <= 0;
12			else
13			case (opcode)
14	1	1240	ADD: out <= operand1 + operand2;
15	1	1180	SUB: out <= operand1 - operand2;
16	1	1232	MULT: out <= operand1 * operand2;
17	1	1357	DIV: out <= operand1 / operand2;

code_cvr_alu - Notepad
File Edit Format View Help

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	52	52	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /tb/DUT --

	Node	1H->0L	0L->1H	"Coverage"
	-----	-----	-----	-----
	clk	1	1	100.00
	operand1[7-0]	1	1	100.00
	operand2[7-0]	1	1	100.00
	out[7-0]	1	1	100.00
	rst	1	1	100.00
Total Node Count = 26				
Toggled Node Count = 26				
Untoggled Node Count = 0				
Toggle Coverage = 100.00% (52 of 52 bins)				

8. Functional Coverage report snippets (Questions 1, 2, and 3 only)

Covergroups									
File Bookmarks Window Help									
Covergroups									
Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/testing_pkg/Transaction		100.00%							
TYPE CovCode		100.00%	100	100.00...		✓			auto(0)
CVP CovCode::operand1_cp		100.00%	100	100.00...		✓			
CVP CovCode::operand2_cp		100.00%	100	100.00...		✓			
CVP CovCode::opcode_cp		100.00%	100	100.00...		✓			
CROSS CovCode::opcode_operand1		100.00%	100	100.00...		✓			
CROSS CovCode::opcode_operand2		100.00%	100	100.00...		✓			
INST \testing_pkg::Transaction::CovCode		100.00%	100	100.00...		✓			0
CVP operand1_cp		100.00%	100	100.00...		✓			
bin max_neg		44	1	100.00...		✓			
bin zero		48	1	100.00...		✓			
bin max_pos		44	1	100.00...		✓			
default bin misc		9874	-	-		✓			
CVP operand2_cp		100.00%	100	100.00...		✓			
bin max_neg		38	1	100.00...		✓			
bin zero		50	1	100.00...		✓			
bin max_pos		52	1	100.00...		✓			
default bin misc		9870	-	-		✓			
CVP opcode_cp		100.00%	100	100.00...		✓			
illegal_bin no_div		2696	-	-		✓			
bin add_sub		4850	1	100.00...		✓			
bin add_mult		4954	1	100.00...		✓			
bin mult_sub		4824	1	100.00...		✓			
bin add_then_sub		270	1	100.00...		✓			
bin add_then_mult		268	1	100.00...		✓			
bin mult_then_sub		294	1	100.00...		✓			
CROSS opcode_operand1		100.00%	100	100.00...		✓			
bin max_pos_add_or_sub		18	1	100.00...		✓			
bin max_pos_add_or_mult		20	1	100.00...		✓			
bin max_pos_mult_or_sub		18	1	100.00...		✓			
bin max_neg_add_or_sub		22	1	100.00...		✓			
bin max_neg_add_or_mult		26	1	100.00...		✓			
bin max_neg_mult_or_sub		20	1	100.00...		✓			
CROSS opcode_operand2		100.00%	100	100.00...		✓			
bin max_pos_add_or_sub		28	1	100.00...		✓			
bin max_pos_add_or_mult		14	1	100.00...		✓			
bin max_pos_mult_or_sub		26	1	100.00...		✓			
bin max_neg_add_or_sub		10	1	100.00...		✓			
bin max_neg_add_or_mult		24	1	100.00...		✓			
bin max_neg_mult_or_sub		26	1	100.00...		✓			

code_cvr_alu - Notepad

File Edit Format View Help

Covergroup Coverage:

Covergroups	1	na	na	100.00%
Coverpoints/Crosses	5	na	na	na
Covergroup Bins	24	24	0	100.00%

Covergroup	Metric	Goal	Bins	Status
TYPE /testing_pkg/Transaction/CovCode	100.00%	100	-	Covered
covered/total bins:	24	24	-	
missing/total bins:	0	24	-	
% Hit:	100.00%	100	-	
Coverpoint operand1_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint operand2_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint opcode_cp	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Cross opcode_operand1	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Cross opcode_operand2	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Covergroup instance \/testing_pkg::Transaction::CovCode	100.00%	100	-	Covered
covered/total bins:	24	24	-	
missing/total bins:	0	24	-	
% Hit:	100.00%	100	-	
Coverpoint operand1_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin max_neg	44	1	-	Covered
bin zero	48	1	-	Covered

code_cvr_alu - Notepad

File Edit Format View Help

bin zero	48	1	-	Covered
bin max_pos	44	1	-	Covered
default bin misc	9874		-	Occurred
Coverpoint operand2_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin max_neg	38	1	-	Covered
bin zero	50	1	-	Covered
bin max_pos	52	1	-	Covered
default bin misc	9870		-	Occurred
Coverpoint opcode_cp	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
illegal_bin no_div	2696		-	Occurred
bin add_sub	4850	1	-	Covered
bin add_mult	4954	1	-	Covered
bin mult_sub	4824	1	-	Covered
bin add_then_sub	270	1	-	Covered
bin add_then_mult	268	1	-	Covered
bin mult_then_sub	294	1	-	Covered
Cross opcode_operand1	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin max_pos_add_or_sub	18	1	-	Covered
bin max_pos_add_or_mult	20	1	-	Covered
bin max_pos_mult_or_sub	18	1	-	Covered
bin max_neg_add_or_sub	22	1	-	Covered
bin max_neg_add_or_mult	26	1	-	Covered
bin max_neg_mult_or_sub	20	1	-	Covered
Cross opcode_operand2	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin max_pos_add_or_sub	28	1	-	Covered
bin max_pos_add_or_mult	14	1	-	Covered
bin max_pos_mult_or_sub	26	1	-	Covered
bin max_neg_add_or_sub	10	1	-	Covered
bin max_neg_add_or_mult	24	1	-	Covered

code_cvr_alu - Notepad

File Edit Format View Help

Cross opcode_operand2	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin max_pos_add_or_sub	28	1	-	Covered
bin max_pos_add_or_mult	14	1	-	Covered
bin max_pos_mult_or_sub	26	1	-	Covered
bin max_neg_add_or_sub	10	1	-	Covered
bin max_neg_add_or_mult	24	1	-	Covered
bin max_neg_mult_or_sub	26	1	-	Covered

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	1	1	0	100.00%

.....

code_cvr_alu - Notepad

File Edit Format View Help

COVERGROUP COVERAGE:

Covergroup	Metric	Goal	Bins	Status
TYPE /testing_pkg/Transaction/CovCode	100.00%	100	-	Covered
covered/total bins:	24	24	-	
missing/total bins:	0	24	-	
% Hit:	100.00%	100	-	
Coverpoint operand1_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint operand2_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
Coverpoint opcode_cp	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Cross opcode_operand1	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Cross opcode_operand2	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Covergroup instance \testing_pkg::Transaction::CovCode	100.00%	100	-	Covered
covered/total bins:	24	24	-	
missing/total bins:	0	24	-	
% Hit:	100.00%	100	-	
Coverpoint operand1_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin max_neg	44	1	-	Covered
bin zero	48	1	-	Covered
bin max_pos	44	1	-	Covered
default bin misc	9874		-	Occurred
Coverpoint operand2_cp	100.00%	100	-	Covered

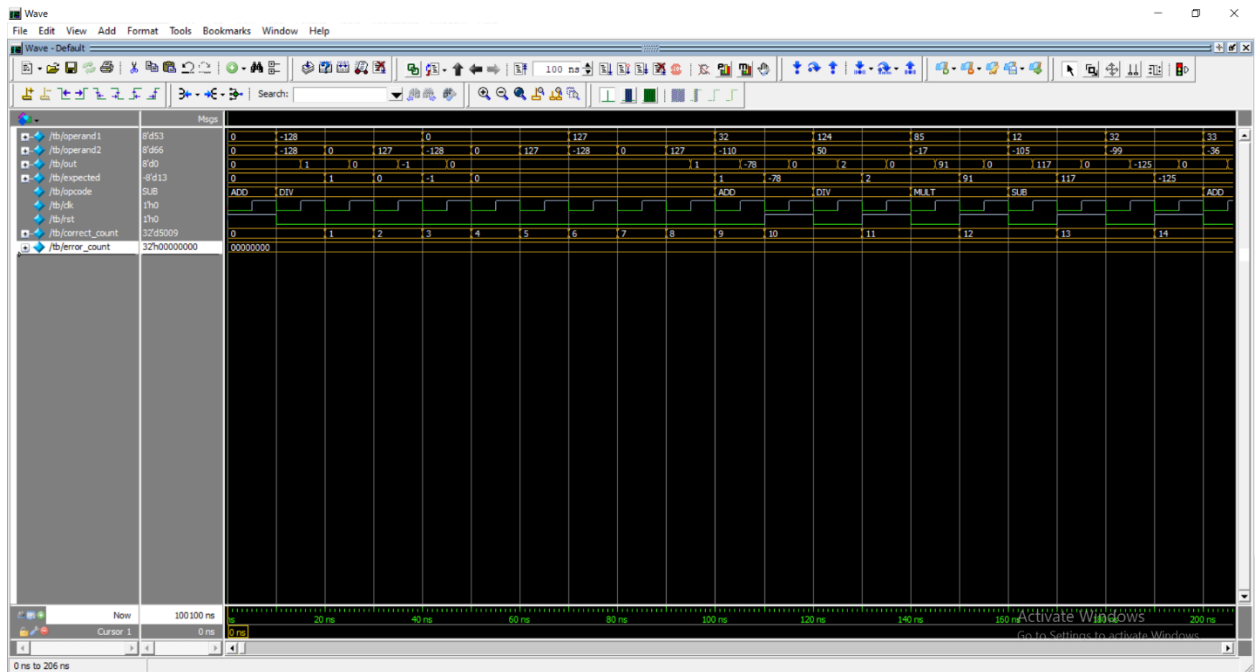
code_cvr_alu - Notepad

File Edit Format View Help

Coverpoint operand2_cp	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin max_neg	38	1	-	Covered
bin zero	50	1	-	Covered
bin max_pos	52	1	-	Covered
default bin misc	9870		-	Occurred
Coverpoint opcode_cp	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
illegal_bin no_div	2696		-	Occurred
bin add_sub	4850	1	-	Covered
bin add_mult	4954	1	-	Covered
bin mult_sub	4824	1	-	Covered
bin add_then_sub	270	1	-	Covered
bin add_then_mult	268	1	-	Covered
bin mult_then_sub	294	1	-	Covered
Cross opcode_operand1	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin max_pos_add_or_sub	18	1	-	Covered
bin max_pos_add_or_mult	20	1	-	Covered
bin max_pos_mult_or_sub	18	1	-	Covered
bin max_neg_add_or_sub	22	1	-	Covered
bin max_neg_add_or_mult	26	1	-	Covered
bin max_neg_mult_or_sub	20	1	-	Covered
Cross opcode_operand2	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
Auto, Default and User Defined Bins:				
bin max_pos_add_or_sub	28	1	-	Covered
bin max_pos_add_or_mult	14	1	-	Covered
bin max_pos_mult_or_sub	26	1	-	Covered
bin max_neg_add_or_sub	10	1	-	Covered
bin max_neg_add_or_mult	24	1	-	Covered
bin max_neg_mult_or_sub	26	1	-	Covered

TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 1

9. Clear and neat QuestaSim waveform snippets showing the functionality of the design



Q2

1. Testbench code

```

1  import question_2_pkg::*;
2
3  module question_2_tb();
4      parameter int WIDTH = question_2_pkg::WIDTH;
5
6      // =====
7      // DUT signals
8      // =====
9      logic clk;
10     logic rst_n, load_n, up_down, ce;
11     logic [WIDTH-1:0] data_load;
12     logic [WIDTH-1:0] count_out;
13     logic max_count, zero;
14
15     // expected
16     logic [WIDTH-1:0] count_out_expected;
17     bit expected_max;
18     bit expected_zero;
19
20     // class object
21     question_2_class question_2_object;
22
23     // =====
24     // DUT instantiation
25     // =====
26     counter #(.WIDTH(WIDTH)) dut (
27         .clk(clk),
28         .rst_n(rst_n),
29         .load_n(load_n),
30         .up_down(up_down),
31         .ce(ce),
32         .data_load(data_load),
33         .count_out(count_out),
34         .max_count(max_count),
35         .zero(zero)
36     );

```

```

38 // =====
39 // Clock generation
40 // =====
41 initial begin
42     clk = 0;
43     forever #5 clk = ~clk; // 10ns period
44 end
45
46 // =====
47 // Tasks
48 // =====
49 task assert_reset();
50     begin
51         rst_n = 0;
52         @(posedge clk);
53         rst_n = 1;
54         @(posedge clk);
55     end
56 endtask
57
58 task golden_model (
59     input logic [WIDTH-1:0] prev_count,
60     input bit load_n_in, ce_in, up_down_in,
61     input logic [WIDTH-1:0] data_load_in,
62     output logic [WIDTH-1:0] expected,
63     output bit expected_max_o,
64     output bit expected_zero_o
65 );
66     if (!load_n_in)
67         expected = data_load_in;
68     else if (ce_in)
69         expected = up_down_in ? prev_count + 1 : prev_count - 1;
70     else
71         expected = prev_count;
72

```

```

72
73     expected_max_o = (expected == {WIDTH{1'b1}});
74     expected_zero_o = (expected == 0);
75 endtask
76
77 task check_result(
78     input logic [WIDTH-1:0] expected,
79     input bit expected_max_i,
80     input bit expected_zero_i
81 );
82     if (count_out != expected)
83         $display("ERROR @%0t: count_out mismatch. got=%0d expected=%0d",
84             $time, count_out, expected);
85     if (max_count != expected_max_i)
86         $display("ERROR @%0t: max_count mismatch. got=%b expected=%b",
87             $time, max_count, expected_max_i);
88     if (zero != expected_zero_i)
89         $display("ERROR @%0t: zero mismatch. got=%b expected=%b",
90             $time, zero, expected_zero_i);
91     if ((count_out === expected) &&
92         (max_count === expected_max_i) &&
93         (zero === expected_zero_i))
94         $display("PASS @%0t: count_out=%0d, max=%b, zero=%b",
95             $time, count_out, max_count, zero);
96 endtask
97

```

```

98 // =====
99 // Stimulus
100 // =====
101 initial begin
102     // init signals
103     rst_n    = 1'b1;
104     load_n   = 1'b1;
105     ce       = 1'b0;
106     up_down  = 1'b1;
107     data_load = '0;
108
109     count_out_expected = '0;
110     expected_max    = 1'b0;
111     expected_zero    = 1'b1;
112
113     // create class object
114     question_2_object = new();
115
116     // apply reset
117     assert_reset();
118
119     // run randomized tests
120     repeat (40000) begin
121
122         // save previous count before DUT update
123         static logic [WIDTH-1:0] prev_count = count_out;
124
125         // randomize fields in class
126         assert(question_2_object.randomize())
127             else $fatal("Randomization failed!");
128

```



```

129 // drive DUT inputs
130 load_n = question_2_object.load_n_class;
131 ce      = question_2_object.ce_class;
132 up_down = question_2_object.up_down_class;
133 data_load = question_2_object.data_load_class;
134
135 // wait for DUT update
136 @(posedge clk);
137
138 // compute expected output
139 golden_model(prev_count, load_n, ce, up_down, data_load,
140             count_out_expected, expected_max, expected_zero);
141
142 // sample coverage
143 question_2_object.sampled_count = count_out;
144 question_2_object.counter_cov.sample();
145
146 // check result
147 check_result(count_out_expected, expected_max, expected_zero);
148 end
149
150 $display("=== TEST COMPLETE ===");
151 $finish;
152 end
153
154 endmodule
155

```

2. Package code

```

1 package question_2_pkg;
2     parameter int WIDTH = 4;
3
4     class question_2_class;
5         // random control fields
6         rand logic rst_n_class, load_n_class, up_down_class, ce_class;
7         rand logic [WIDTH-1:0] data_load_class;
8
9         // variable that TB will fill with DUT count for coverage sampling
10        logic [WIDTH-1:0] sampled_count;
11
12        // Constraints / distributions
13        constraint distribution {
14            rst_n_class dist {1 := 80, 0 := 20};
15            load_n_class dist {0 := 70, 1 := 30};
16            ce_class dist {1 := 70, 0 := 30};
17        }
18
19        // Covergroup defined inside class (no event control here)
20        covergroup counter_cov;
21            // 1. load-data when load asserted and reset deasserted
22            load_data_cp: coverpoint data_load_class
23                iff (load_n_class == 0 && rst_n_class == 1);
24
25            // 2. count-up values (sampled_count) when rst=1, ce=1, up_down=1
26            count_up_cp: coverpoint sampled_count
27                iff (rst_n_class == 1 && ce_class == 1 && up_down_class == 1);
28
29            // 3. overflow detection (max -> 0) as transition bin
30            count_up_overflow: coverpoint sampled_count
31                iff (rst_n_class == 1 && ce_class == 1 && up_down_class == 1) {
32                bins overflow = ({WIDTH{1'b1}} => 0);
33            }
34

```

```

34
35     // 4. count-down values when rst=1, ce=1, up_down=0
36     count_down_cp: coverpoint sampled_count
37     iff (rst_n_class == 1 && ce_class == 1 && up_down_class == 0);
38
39     // 5. underflow detection (0 -> max) as transition bin
40     count_down_underflow: coverpoint sampled_count
41     iff (rst_n_class == 1 && ce_class == 1 && up_down_class == 0) {
42         bins underflow = (0 => {WIDTH{1'b1}});
43     }
44 endgroup
45
46 function new();
47     counter_cov = new();
48 endfunction
49
50 endclass
51
52 endpackage
53

```

3. Design code

```

1  ///////////////////////////////////////////////////////////////////
2  // Author: Kareem Waseem
3  // Course: Digital Verification using SV & UVM
4  //
5  // Description: Counter Design
6  //
7  ///////////////////////////////////////////////////////////////////
8  module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
9  parameter WIDTH = 4;
10 input clk;
11 input rst_n;
12 input load_n;
13 input up_down;
14 input ce;
15 input [WIDTH-1:0] data_load;
16 output reg [WIDTH-1:0] count_out;
17 output max_count;
18 output zero;
19
20 always @(posedge clk) begin
21     if (!rst_n)
22         count_out <= 0;
23     else if (!load_n)
24         count_out <= data_load;
25     else if (ce)
26         if (up_down)
27             count_out <= count_out + 1;
28         else
29             count_out <= count_out - 1;
30 end
31
32 assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
33 assign zero = (count_out == 0)? 1:0;
34
35 endmodule

```

4. Report any bugs detected in the design and fix them

5. Snippet to your verification plan document

Label	Design requirement description	Stimulus generation	Functional coverage	Functionality check
COUNTER_RESET	When reset is asserted, count_out = 0.	Directed at start	Coverpoint implicitly skipped since reset is 0.	Task compares DUT output with expected=0.
COUNTER_LOAD	When load is asserted, count_out = data_load.	Random + Directed corners	load_data_cp coverpoint samples data_load when load_n=0, rst_n=1.	Task golden_model ensures DUT output matches data_load.
COUNTER_COUNT_UP	When up_down=1, enable=1, DUT counts upward.	Random + Directed	count_up_cp auto-bins all values of count_out.	Task golden_model + check_result.
COUNTER_OVERFLOW	When counting up, after max value counter wraps around to 0.	Directed (set to max, ce=1)	count_up_overflow transition bin checks {max → 0}.	Checker compares expected wraparound.
COUNTER_COUNT_DOWN	When up_down=0, enable=1, DUT counts downward.	Random + Directed	count_down_cp auto-bins all values of count_out.	Task golden_model + check_result.
COUNTER_UNDERFLOW	When counting down, after 0 the counter wraps to max.	Directed (set to 0, ce=1)	count_down_underflow transition bin checks {0 → max}.	Checker compares expected wraparound.
COUNTER_ENABLE_HOLD	If enable=0, count_out must hold its previous value.	Random	Can extend coverage with cross of ce=0 & count_out.	Task ensures DUT output equals previous value.

6. Do file

```

1  vlib work
2  vlog counter.v package.svh question_2_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.question_2_tb -cover
4  add wave *
5  coverage save question_2_tb.ucdb -onexit
6  run -all

```

7. Code Coverage report snippets (Questions 1, 2, and 3 only)

Coverage Report by instance with details

```
=====
=== Instance: /question_2_tb/dut
=== Design Unit: work.counter
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	10	10	0	100.00%

```
=====Branch Details=====
```

```
Branch Coverage for instance /question_2_tb/dut
```

Line	Item	Count	Source
----	----	----	-----
File counter.v			
-----IF Branch-----			
21		39966	Count coming in to IF
21	1	1	if (!rst_n)
23	1	27873	else if (!load_n)
25	1	8483	else if (ce)
		3609	All False Count

```
Branch totals: 4 hits of 4 branches = 100.00%
```

-----IF Branch-----			
26		8483	Count coming in to IF
26	1	4274	if (up_down)
28	1	4209	else

```
Branch totals: 2 hits of 2 branches = 100.00%
```

-----IF Branch-----			
32		34590	Count coming in to IF
32	1	2163	assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
32	2	32427	assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
33                               34590    Count coming in to IF
33              1                2182    assign zero = (count_out == 0)? 1:0;

33              2                32408    assign zero = (count_out == 0)? 1:0;
```

Branch totals: 2 hits of 2 branches = 100.00%

Condition Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
-----	----	----	----	-----
Conditions	2	2	0	100.00%

=====Condition Details=====

Condition Coverage for instance /question_2_tb/dut --

File counter.v

-----Focused Condition View-----

Line 32 Item 1 (count_out == {4{{1}}})
Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(count_out == {4{{1}}})	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(count_out == {4{{1}}})_0	-
Row 2:	1	(count_out == {4{{1}}})_1	-

-----Focused Condition View-----

Line 33 Item 1 (count_out == 0)
Condition totals: 1 of 1 input term covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
(count_out == 0)	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	(count_out == 0)_0	-
Row 2:	1	(count_out == 0)_1	-

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Statements	7	7	0	100.00%

=====Statement Details=====

Statement Coverage for instance /question_2_tb/dut --

Line	Item	Count	Source
8	File counter.v		module counter (clk ,rst_n, load_n, up_down, ce, data_load, count_out, max_count, zero);
9			parameter WIDTH = 4;
10			input clk;
11			input rst_n;
12			input load_n;
13			input up_down;
14			input ce;
15			input [WIDTH-1:0] data_load;
16			output reg [WIDTH-1:0] count_out;
17			output max_count;
18			output zero;
19			


```

20          1          39966    always @(posedge clk) begin
21                                if (!rst_n)
22          1          1          count_out <= 0;
23                                else if (!load_n)
24          1          27873      count_out <= data_load;
25                                else if (ce)
26                                if (up_down)
27          1          4274      count_out <= count_out + 1;
28                                else
29          1          4209      count_out <= count_out - 1;
30                                end
31
32          1          34591    assign max_count = (count_out == {WIDTH{1'b1}})? 1:0;
33          1          34591    assign zero = (count_out == 0)? 1:0;

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	28	28	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /question_2_tb/dut --

Node	1H->0L	0L->1H	"Coverage"
ce	1	1	100.00
clk	1	1	100.00

code_cvr_alu - Notepad

File Edit Format View Help

Toggles	28	28	0	100.00%
---------	----	----	---	---------

=====Toggle Details=====

Toggle Coverage for instance /question_2_tb/dut --

Node	1H->0L	0L->1H	"Coverage"
ce	1	1	100.00
clk	1	1	100.00
count_out[3-0]	1	1	100.00
data_load[0-3]	1	1	100.00
load_n	1	1	100.00
max_count	1	1	100.00
up_down	1	1	100.00
zero	1	1	100.00

Total Node Count = 14

Toggled Node Count = 14

Untoggled Node Count = 0

Toggle Coverage = 100.00% (28 of 28 bins)

=====

8. Functional Coverage report snippets (Questions 1, 2, and 3 only)

Covergroups

File Bookmarks Window Help

Covergroups

Name	Class Type	Coverage	Goal	% of Goal	Status	Included
[-] /question_2_pkg/question_2_class		100.00%				
[-] TYPE counter_cov		100.00%	100	100.00...		✓
CVP counter_cov::load_data_cp		100.00%	100	100.00...		✓
CVP counter_cov::count_up_cp		100.00%	100	100.00...		✓
CVP counter_cov::count_up_overflow		100.00%	100	100.00...		✓
CVP counter_cov::count_down_cp		100.00%	100	100.00...		✓
CVP counter_cov::count_down_underflow		100.00%	100	100.00...		✓
[-] INST \question_2_pkg::question_2_class::counter_cov		100.00%	100	100.00...		✓
+ CVP load_data_cp		100.00%	100	100.00...		✓
+ CVP count_up_cp		100.00%	100	100.00...		✓
+ CVP count_up_overflow		100.00%	100	100.00...		✓
+ CVP count_down_cp		100.00%	100	100.00...		✓
+ CVP count_down_underflow		100.00%	100	100.00...		✓

Covergroup Coverage:

Covergroups	1	na	na	100.00%
Coverpoints/Crosses	5	na	na	na
Covergroup Bins	50	50	0	100.00%

Covergroup	Metric	Goal	Bins	Status
TYPE /question_2_pkg/question_2_class/counter_cov	100.00%	100	-	Covered
covered/total bins:	50	50	-	
missing/total bins:	0	50	-	
% Hit:	100.00%	100	-	
Coverpoint load_data_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
Coverpoint count_up_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
Coverpoint count_up_overflow	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
Coverpoint count_down_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
Coverpoint count_down_underflow	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
Covergroup instance \question_2_pkg::question_2_class::counter_cov	100.00%	100	-	Covered
covered/total bins:	50	50	-	
missing/total bins:	0	50	-	
% Hit:	100.00%	100	-	
Coverpoint load_data_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
bin auto[0]	1447	1	-	Covered

code_cvr_alu - Notepad

File Edit Format View Help

bin auto[0]	1447	1	-	Covered
bin auto[1]	1408	1	-	Covered
bin auto[2]	1390	1	-	Covered
bin auto[3]	1391	1	-	Covered
bin auto[4]	1329	1	-	Covered
bin auto[5]	1322	1	-	Covered
bin auto[6]	1328	1	-	Covered
bin auto[7]	1413	1	-	Covered
bin auto[8]	1417	1	-	Covered
bin auto[9]	1377	1	-	Covered
bin auto[10]	1396	1	-	Covered
bin auto[11]	1308	1	-	Covered
bin auto[12]	1411	1	-	Covered
bin auto[13]	1429	1	-	Covered
bin auto[14]	1409	1	-	Covered
bin auto[15]	1375	1	-	Covered
Coverpoint count_up_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
bin auto[0]	695	1	-	Covered
bin auto[1]	701	1	-	Covered
bin auto[2]	687	1	-	Covered
bin auto[3]	718	1	-	Covered
bin auto[4]	640	1	-	Covered
bin auto[5]	644	1	-	Covered
bin auto[6]	700	1	-	Covered
bin auto[7]	714	1	-	Covered
bin auto[8]	696	1	-	Covered
bin auto[9]	690	1	-	Covered
bin auto[10]	718	1	-	Covered
bin auto[11]	682	1	-	Covered
bin auto[12]	699	1	-	Covered
bin auto[13]	727	1	-	Covered
bin auto[14]	749	1	-	Covered
bin auto[15]	686	1	-	Covered
Coverpoint count_up_overflow	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
bin overflow	94	1	-	Covered
Coverpoint count_down_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	

code_cvr_alu - Notepad

File Edit Format View Help

covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
bin auto[0]	732	1	-	Covered
bin auto[1]	718	1	-	Covered
bin auto[2]	706	1	-	Covered
bin auto[3]	678	1	-	Covered
bin auto[4]	683	1	-	Covered
bin auto[5]	645	1	-	Covered
bin auto[6]	648	1	-	Covered
bin auto[7]	671	1	-	Covered
bin auto[8]	708	1	-	Covered
bin auto[9]	680	1	-	Covered
bin auto[10]	698	1	-	Covered
bin auto[11]	659	1	-	Covered
bin auto[12]	736	1	-	Covered
bin auto[13]	737	1	-	Covered
bin auto[14]	661	1	-	Covered
bin auto[15]	710	1	-	Covered
Coverpoint count_down_underflow	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
bin underflow	97	1	-	Covered

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	1	1	0	100.00%

2017-08-10 10:11

COVERGROUP COVERAGE:

Covergroup	Metric	Goal	Bins	Status
TYPE /question_2_pkg/question_2_class/counter_cov	100.00%	100	-	Covered
covered/total bins:	50	50	-	
missing/total bins:	0	50	-	
% Hit:	100.00%	100	-	
Coverpoint load_data_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
Coverpoint count_up_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
Coverpoint count_up_overflow	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
Coverpoint count_down_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
Coverpoint count_down_underflow	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
Covergroup instance \question_2_pkg::question_2_class::counter_cov	100.00%	100	-	Covered
covered/total bins:	50	50	-	
missing/total bins:	0	50	-	
% Hit:	100.00%	100	-	
Coverpoint load_data_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
bin auto[0]	1447	1	-	Covered
bin auto[1]	1408	1	-	Covered
bin auto[2]	1390	1	-	Covered
bin auto[3]	1391	1	-	Covered
bin auto[4]	1329	1	-	Covered

code_cvr_alu - Notepad

File Edit Format View Help

bin auto[4]	1329	1	-	Covered
bin auto[5]	1322	1	-	Covered
bin auto[6]	1328	1	-	Covered
bin auto[7]	1413	1	-	Covered
bin auto[8]	1417	1	-	Covered
bin auto[9]	1377	1	-	Covered
bin auto[10]	1396	1	-	Covered
bin auto[11]	1308	1	-	Covered
bin auto[12]	1411	1	-	Covered
bin auto[13]	1429	1	-	Covered
bin auto[14]	1409	1	-	Covered
bin auto[15]	1375	1	-	Covered
Coverpoint count_up_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
bin auto[0]	695	1	-	Covered
bin auto[1]	701	1	-	Covered
bin auto[2]	687	1	-	Covered
bin auto[3]	718	1	-	Covered
bin auto[4]	640	1	-	Covered
bin auto[5]	644	1	-	Covered
bin auto[6]	700	1	-	Covered
bin auto[7]	714	1	-	Covered
bin auto[8]	696	1	-	Covered
bin auto[9]	690	1	-	Covered
bin auto[10]	718	1	-	Covered
bin auto[11]	682	1	-	Covered
bin auto[12]	699	1	-	Covered
bin auto[13]	727	1	-	Covered
bin auto[14]	749	1	-	Covered
bin auto[15]	686	1	-	Covered
Coverpoint count_up_overflow	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
bin overflow	94	1	-	Covered
Coverpoint count_down_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
bin auto[0]	732	1	-	Covered
bin auto[1]	718	1	-	Covered

code_cvr_alu - Notepad

File Edit Format View Help

covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
bin overflow	94	1	-	Covered
Coverpoint count_down_cp	100.00%	100	-	Covered
covered/total bins:	16	16	-	
missing/total bins:	0	16	-	
% Hit:	100.00%	100	-	
bin auto[0]	732	1	-	Covered
bin auto[1]	718	1	-	Covered
bin auto[2]	706	1	-	Covered
bin auto[3]	678	1	-	Covered
bin auto[4]	683	1	-	Covered
bin auto[5]	645	1	-	Covered
bin auto[6]	648	1	-	Covered
bin auto[7]	671	1	-	Covered
bin auto[8]	708	1	-	Covered
bin auto[9]	680	1	-	Covered
bin auto[10]	698	1	-	Covered
bin auto[11]	659	1	-	Covered
bin auto[12]	736	1	-	Covered
bin auto[13]	737	1	-	Covered
bin auto[14]	661	1	-	Covered
bin auto[15]	710	1	-	Covered
Coverpoint count_down_underflow	100.00%	100	-	Covered
covered/total bins:	1	1	-	
missing/total bins:	0	1	-	
% Hit:	100.00%	100	-	
bin underflow	97	1	-	Covered

TOTAL COVERGROUP COVERAGE: 100.00% COVERGROUP TYPES: 1

Q3

1. Testbench code

```

1
2 import alsu_pkg::*;
3
4 module question_3_tb;
5
6     // DUT Signals
7     logic clk, rst;
8     logic signed [2:0] A, B;
9     logic cin, serial_in;
10    logic red_op_A, red_op_B;
11    logic bypass_A, bypass_B;
12    logic direction;
13    opcode_e opcode;
14    logic signed [5:0] result;
15
16    // Transaction object
17    alsu_txn t;
18
19    // Instantiate DUT
20    ALSU DUT (
21        .clk(clk),
22        .rst(rst),
23        .A(A),
24        .B(B),
25        .cin(cin),
26        .serial_in(serial_in),
27        .red_op_A(red_op_A),
28        .red_op_B(red_op_B),
29        .bypass_A(bypass_A),
30        .bypass_B(bypass_B),
31        .direction(direction),
32        .opcode(opcode),
33        .out[result]
34    );
35

```

```

36 // Clock generation
37 always #5 clk = ~clk;
38
39 initial begin
40     clk = 0;
41     rst = 1;
42     #20 rst = 0;
43
44     t = new();
45
46     // ----- First Loop -----
47     t.en_c8 = 0; // disable constraint #8
48     repeat (1000) begin
49         if (!t.randomize()) begin
50             $display("Randomization failed in first loop!");
51         end
52
53         // Drive DUT inputs
54         A = t.A;
55         B = t.B;
56         cin = t.cin;
57         serial_in = t.serial_in;
58         red_op_A = t.red_op_A;
59         red_op_B = t.red_op_B;
60         bypass_A = t.bypass_A;
61         bypass_B = t.bypass_B;
62         direction = t.direction;
63         opcode = t.opcode;
64
65         // Coverage sampling condition
66         if (!rst && !bypass_A && !bypass_B) begin
67             t.cvr_gp.sample();
68         end
69

```

```

70     #10;
71 end
72
73 // ----- Second Loop -----
74 $display("Starting second loop for opcode sequence...");
75 rst = 0;
76 bypass_A = 0;
77 bypass_B = 0;
78 red_op_A = 0;
79 red_op_B = 0;
80
81 t.en_c1 = 0;
82 t.en_c2 = 0;
83 t.en_c3 = 0;
84 t.en_c8 = 1; // enable constraint #8
85
86 // Generate unique opcode sequence
87 if (!t.randomize()) begin
88     $display("Randomization failed for opcode sequence!");
89 end
90
91 // Iterate through the unique opcode sequence
92 foreach (t.opcode_seq[i]) begin
93     opcode = t.opcode_seq[i];
94
95     repeat (6) begin
96         if (!t.randomize()) begin
97             $display("Randomization failed inside second loop!");
98         end
99
100         A = t.A;
101         B = t.B;
102         cin = t.cin;

```

```

99
100     A = t.A;
101     B = t.B;
102     cin = t.cin;
103     serial_in = t.serial_in;
104     direction = t.direction;
105
106     if (!rst && !bypass_A && !bypass_B) begin
107         t.cvr_gp.sample();
108     end
109
110     #10;
111 end
112 end
113
114 $display("Simulation completed.");
115 $stop;
116 end
117 endmodule
118

```

2. Package code

```

1  package alsu_pkg;
2
3  // ----- ENUM for opcodes -----
4  typedef enum logic [2:0] {
5      OR_OP      = 3'h0,
6      XOR_OP     = 3'h1,
7      ADD_OP     = 3'h2,
8      MUL_OP     = 3'h3,
9      SHIFT_OP   = 3'h4,
10     ROT_OP     = 3'h5,
11     INVALID_6   = 3'h6,
12     INVALID_7   = 3'h7
13 } opcode_e;
14
15 // ----- Transaction Class -----
16 class alsu_txn;
17
18     // Constants
19     localparam signed [2:0] MAXPOS = 3;
20     localparam signed [2:0] MAXNEG = -4;
21     localparam signed [2:0] ZERO   = 0;
22
23     // Randomizable Inputs
24     rand logic clk, rst, cin, serial_in;
25     rand logic red_op_A, red_op_B;
26     rand logic bypass_A, bypass_B, direction;
27     rand opcode_e opcode;
28     rand logic signed [2:0] A, B;
29
30     // Fixed array for constraint #8
31     rand opcode_e opcode_seq[6];
32
33     // Constraint enable flags
34     bit en_c1, en_c2, en_c3, en_c4, en_c5, en_c6, en_c7, en_c8;
35

```



```

36 // ----- Covergroup -----
37 covergroup cvr_gp @(posedge clk);
38     option.per_instance = 1;
39
40     // A Coverpoint
41     coverpoint A {
42         bins A_data_0      = {0};
43         bins A_data_max    = {MAXPOS};
44         bins A_data_min    = {MAXNEG};
45         bins A_data_default = default;
46     }
47
48     // B Coverpoint
49     coverpoint B {
50         bins B_data_0      = {0};
51         bins B_data_max    = {MAXPOS};
52         bins B_data_min    = {MAXNEG};
53         bins B_data_default = default;
54     }
55
56     // Opcode Coverpoint
57     coverpoint opcode {
58         bins bins_shift[]  = {SHIFT_OP, ROT_OP};
59         bins bins_arith[]  = {ADD_OP, MUL_OP};
60         bins bins_bitwise[] = {OR_OP, XOR_OP};
61         illegal_bins bins_invalid = {INVALID_6, INVALID_7};
62     }
63 endgroup
64

```

```

65 // ----- Constraints -----
66 constraint c1 { if (en_c1) A inside {[MAXNEG:MAXPOS]}; }
67 constraint c2 { if (en_c2) B inside {[MAXNEG:MAXPOS]}; }
68 constraint c3 { if (en_c3) opcode inside {OR_OP, XOR_OP, ADD_OP, MUL_OP, SHIFT_OP, ROT_OP}; }
69
70 // Constraint #8: unique opcodes in array
71 constraint unique_opcodes {
72     if (en_c8) {
73         foreach (opcode_seq[i]) opcode_seq[i] inside {OR_OP, XOR_OP, ADD_OP, MUL_OP, SHIFT_OP, ROT_OP};
74         unique {opcode_seq};
75     }
76 }
77
78 // ----- Constructor -----
79 function new();
80     en_c1 = 1; en_c2 = 1; en_c3 = 1; en_c4 = 1;
81     en_c5 = 1; en_c6 = 1; en_c7 = 1; en_c8 = 0;
82     cvr_gp = new();
83 endfunction
84
85 endclass : alsu_txn
86
87 endpackage : alsu_pkg
88

```

3. Design code

```

1  module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
2  parameter INPUT_PRIORITY = "A";
3  parameter FULL_ADDER = "ON";
4  input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5  input [2:0] opcode;
6  input signed [2:0] A, B;
7  output reg [15:0] leds;
8  output reg signed [5:0] out;
9
10 reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
11 reg signed [1:0] cin_reg;
12 reg [2:0] opcode_reg;
13 reg signed [2:0] A_reg, B_reg;
14
15 wire invalid_red_op, invalid_opcode, invalid;
16
17 //Invalid handling
18 assign invalid_red_op = (red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]);
19 assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
20 assign invalid = invalid_red_op | invalid_opcode;
21
22 //Registering input signals
23 always @(posedge clk or posedge rst) begin
24     if(rst) begin
25         cin_reg <= 0;
26         red_op_B_reg <= 0;
27         red_op_A_reg <= 0;
28         bypass_B_reg <= 0;
29         bypass_A_reg <= 0;
30         direction_reg <= 0;
31         serial_in_reg <= 0;
32         opcode_reg <= 0;
33         A_reg <= 0;
34         B_reg <= 0;
35     end else begin
36         cin_reg <= cin;

```

```

37     red_op_B_reg <= red_op_B;
38     red_op_A_reg <= red_op_A;
39     bypass_B_reg <= bypass_B;
40     bypass_A_reg <= bypass_A;
41     direction_reg <= direction;
42     serial_in_reg <= serial_in;
43     opcode_reg <= opcode;
44     A_reg <= A;
45     B_reg <= B;
46 end
47 end
48
49 //leds output blinking
50 always @(posedge clk or posedge rst) begin
51     if(rst) begin
52         leds <= 0;
53     end else begin
54         if (invalid)
55             leds <= ~leds;
56         else
57             leds <= 0;
58     end
59 end
60
61 //ALU output processing
62 always @(posedge clk or posedge rst) begin
63     if(rst) begin
64         out <= 0;
65     end
66     else begin
67         if (bypass_A_reg && bypass_B_reg)
68             out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
69         else if (bypass_A_reg)

```

```

69     else if (bypass_A_reg)
70         out <= A_reg;
71     else if (bypass_B_reg)
72         out <= B_reg;
73     else if (invalid)
74         out <= 0;
75     else begin
76         case (opcode_reg) //edited design error
77             3'h0: begin
78                 if (red_op_A_reg && red_op_B_reg)
79                     out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
80                 else if (red_op_A_reg)
81                     out <= |A_reg;
82                 else if (red_op_B_reg)
83                     out <= |B_reg;
84                 else
85                     out <= A_reg | B_reg;
86             end
87             3'h1: begin
88                 if (red_op_A_reg && red_op_B_reg)
89                     out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
90                 else if (red_op_A_reg)
91                     out <= ^A_reg;
92                 else if (red_op_B_reg)
93                     out <= ^B_reg;
94                 else
95                     out <= A_reg ^ B_reg;
96             end
97             3'h2: out <= (FULL_ADDER == "ON") ? (A_reg + B_reg + cin_reg) : (A_reg + B_reg); //edited design error
98             3'h3: out <= A_reg * B_reg;
99             3'h4: begin
100                 if (direction_reg)
101                     out <= {out[4:0], serial_in_reg};

```

```

100         if (direction_reg)
101             out <= {out[4:0], serial_in_reg};
102         else
103             out <= {serial_in_reg, out[5:1]};
104     end
105     3'h5: begin
106         if (direction_reg)
107             out <= {out[4:0], out[5]};
108         else
109             out <= {out[0], out[5:1]};
110     end
111     default: out <= 0; //edit for 100% coverage and exclude it
112
113 endcase
114 end
115 end
116 end
117
118 endmodule

```

4. Report any bugs detected in the design and fix them

5. Snippet to your verification plan document

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALSU_1	When rst is asserted, both out and leds should be reset to 0.	Directed at the start of the simulation.	-	A checker in the testbench ensures out == 0 and leds == 0 when rst == 1.
ALSU_2	When bypass_A or bypass_B are asserted, out should directly reflect the corresponding input (A or B).	Randomization of bypass_A and bypass_B.	Cover bins for cases where bypass_A == 1, bypass_B == 1, and both asserted.	Checker ensures out == A or out == B according to priority when bypass is active.
ALSU_3	If both bypass_A and bypass_B are asserted, priority is given to A when INPUT_PRIORITY parameter is "A".	Directed stimulus where both bypass signals are high.	Cover bins for both bypass signals high and INPUT_PRIORITY set to "A".	Checker ensures out == A when both bypass signals are asserted.
ALSU_4	When opcode corresponds to bitwise OR (3'h0), the output must equal A OR B.	Randomized values for A, B, and opcode.	Cover bins for opcode 3'h0.	Checker compares DUT output to A OR B.
ALSU_5	When opcode corresponds to bitwise XOR (3'h1), the output must equal A XOR B.	Randomized values for A, B, and opcode.	Cover bins for opcode 3'h1.	Checker compares DUT output to A XOR B.
ALSU_6	When opcode corresponds to ADD (3'h2), output should be A + B + cin if FULL_ADDER parameter is "ON".	Randomized A, B, and cin with opcode == 3'h2.	Cover bins for ADD operation with different cin values.	Checker compares DUT output to expected A + B + cin.
ALSU_7	When opcode corresponds to MUL (3'h3), output should be A * B.	Randomized A and B with opcode == 3'h3.	Cover bins for MUL operation.	Checker compares DUT output to expected A * B.
ALSU_8	When opcode corresponds to SHIFT (3'h4), out should shift left or right depending on direction and serial_in.	Directed and randomized tests toggling direction and serial_in.	Cover bins for shift left and shift right operations.	Checker verifies shift behavior bit-by-bit.

ALSU_9	When opcode corresponds to ROTATE (3'h5), out should rotate left or right based on direction.	Directed and randomized tests toggling direction.	Cover bins for rotate left and rotate right operations.	Checker verifies rotation behavior bit-by-bit.
ALSU_10	When opcode is invalid (3'h6 or 3'h7), out should be 0 and LEDs should blink.	Directed test using invalid opcodes.	Cover bins for invalid opcode values.	Checker ensures out == 0 and leds toggles when invalid opcode is detected.
ALSU_11	When both red_op_A and red_op_B are asserted, priority is given to A when INPUT_PRIORITY is "A".	Randomized with both reduction signals high.	Cover bins for both reduction signals high.	Checker ensures output matches reduction of A only.
ALSU_12	Verify walking ones patterns for A and B according to red_op_A and red_op_B rules.	Randomized inputs with special handling to generate walking ones 001, 010, 100.	Cover bins for A_data_walkingones [] and B_data_walkingones [].	Checker verifies outputs match reduction rules when walking ones are applied.
ALSU_13	Validate transition of opcodes in the sequence 0 -> 1 -> 2 -> 3 -> 4 -> 5.	Constraint 8 to generate a unique fixed array of opcodes.	Cover transition bins for the specified opcode sequence.	Checker verifies that opcodes occur in the specified sequence and behavior matches expectations.
ALSU_14	Verify LED blinking behavior whenever invalid signal is high.	Directed test toggling invalid condition.	Cover bins for LED state toggling.	Checker verifies LEDs toggle at every clock edge while invalid is high.

6. Do file

```

1  vlib work
2  vlog ALSU.v package_q_3.svh question_3_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.question_3_tb -cover
4  add wave *
5  coverage save question_3_tb.ucdb -onexit |
6  run -all

```

7. Code Coverage report snippets (Questions 1, 2, and 3 only)

Coverage Report by instance with details

```
=====
=== Instance: /question_3_tb/DUT
=== Design Unit: work.ALSU
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	31	31	0	100.00%

=====Branch Details=====

Branch Coverage for instance /question_3_tb/DUT

Line	Item	Count	Source
----	----	----	-----
File ALSU.v			
-----IF Branch-----			
24		1038	Count coming in to IF
24	1	2	if(rst) begin
35	1	1036	end else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----			
51		1039	Count coming in to IF
51	1	3	if(rst) begin
53	1	1036	end else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----			
54		1036	Count coming in to IF
54	1	511	if (invalid)
56	1	525	else

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
63          1038    Count coming in to IF
63          1      2      if(rst) begin

66          1036    else begin

Branch totals: 2 hits of 2 branches = 100.00%

-----IF Branch-----
67          1036    Count coming in to IF
67          1      226   if (bypass_A_reg && bypass_B_reg)

69          262     else if (bypass_A_reg)

71          249     else if (bypass_B_reg)

73          134     else if (invalid)

75          165     else begin

Branch totals: 5 hits of 5 branches = 100.00%

-----CASE Branch-----
76          165    Count coming in to CASE
77          1      50      3'h0: begin

87          44      3'h1: begin

97          15      3'h2: out <= (FULL_ADDER == "ON") ? (A_reg + B_reg + cin_reg) : (A_reg + B_reg); //edited design error

98          15      3'h3: out <= A_reg * B_reg;

99          27      3'h4: begin

105         14      3'h5: begin

Branch totals: 6 hits of 6 branches = 100.00%

-----IF Branch-----
78          50    Count coming in to IF
78          1      7      if (red_op_A_reg && red_op_B_reg)

80          8      else if (red_op_A_reg)
```

84	1	26	else
----	---	----	------

Branch totals: 4 hits of 4 branches = 100.00%

```
-----IF Branch-----
88                                44    Count coming in to IF
88                                10      if (red_op_A_reg && red_op_B_reg)
                                13      else if (red_op_A_reg)
90                                9       else if (red_op_B_reg)
92                                12      else
94
```

Branch totals: 4 hits of 4 branches = 100.00%

```
-----IF Branch-----
100                               27    Count coming in to IF
100                               13      if (direction_reg)
102                               14      else
```

Branch totals: 2 hits of 2 branches = 100.00%

```
-----IF Branch-----
106                               14    Count coming in to IF
106                               10      if (direction_reg)
108                               4       else
```

Branch totals: 2 hits of 2 branches = 100.00%

Condition Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
-----	----	----	-----	-----
Conditions	6	6	0	100.00%

=====Condition Details=====

Condition Coverage for instance /question_3_tb/DUT --

code_cvr_alu - Notepad

File Edit Format View Help

=====Condition Details=====

Condition Coverage for instance /question_3_tb/DUT --

File ALSU.v

-----Focused Condition View-----

Line 67 Item 1 (bypass_A_reg && bypass_B_reg)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
bypass_A_reg	Y		
bypass_B_reg	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	bypass_A_reg_0	-
Row 2:	1	bypass_A_reg_1	bypass_B_reg
Row 3:	1	bypass_B_reg_0	bypass_A_reg
Row 4:	1	bypass_B_reg_1	bypass_A_reg

-----Focused Condition View-----

Line 78 Item 1 (red_op_A_reg && red_op_B_reg)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
red_op_A_reg	Y		
red_op_B_reg	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	red_op_A_reg_0	-
Row 2:	1	red_op_A_reg_1	red_op_B_reg
Row 3:	1	red_op_B_reg_0	red_op_A_reg
Row 4:	1	red_op_B_reg_1	red_op_A_reg

-----Focused Condition View-----

Line 88 Item 1 (red_op_A_reg && red_op_B_reg)

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
------------	---------	------------------------	------

code_cvr_alu - Notepad

File Edit Format View Help

Condition totals: 2 of 2 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
red_op_A_reg	Y		
red_op_B_reg	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	red_op_A_reg_0	-
Row 2:	1	red_op_A_reg_1	red_op_B_reg
Row 3:	1	red_op_B_reg_0	red_op_A_reg
Row 4:	1	red_op_B_reg_1	red_op_A_reg

Expression Coverage:

Enabled Coverage	Bins	Covered	Misses	Coverage
Expressions	8	5	3	62.50%

====Expression Details=====

Expression Coverage for instance /question_3_tb/DUT --

File ALSU.v

-----Focused Expression View-----

Line 18 Item 1 ((red_op_A_reg | red_op_B_reg) & (opcode_reg[1] | opcode_reg[2]))

Expression totals: 4 of 4 input terms covered = 100.00%

Input Term	Covered	Reason for no coverage	Hint
red_op_A_reg	Y		
red_op_B_reg	Y		
opcode_reg[1]	Y		
opcode_reg[2]	Y		

Rows:	Hits	FEC Target	Non-masking condition(s)
Row 1:	1	red_op_A_reg_0	((opcode_reg[1] opcode_reg[2]) && ~red_op_B_reg)
Row 2:	1	red_op_A_reg_1	((opcode_reg[1] opcode_reg[2]) && ~red_op_B_reg)
Row 3:	1	red_op_B_reg_0	((opcode_reg[1] opcode_reg[2]) && ~red_op_A_reg)
Row 4:	1	red_op_B_reg_1	((opcode_reg[1] opcode_reg[2]) && ~red_op_A_reg)
Row 5:	1	opcode_reg[1]_0	((red_op_A_reg red_op_B_reg) && ~opcode_reg[2])

code_cvr_alu - Notepad

File Edit Format View Help

ROW	5:	1	opcode_reg[1]_0	((red_op_A_reg red_op_B_reg) && ~opcode_reg[2])
ROW	6:	1	opcode_reg[1]_1	((red_op_A_reg red_op_B_reg) && ~opcode_reg[2])
ROW	7:	1	opcode_reg[2]_0	((red_op_A_reg red_op_B_reg) && ~opcode_reg[1])
ROW	8:	1	opcode_reg[2]_1	((red_op_A_reg red_op_B_reg) && ~opcode_reg[1])

-----Focused Expression View-----

Line 19 Item 1 (opcode_reg[1] & opcode_reg[2])

Expression totals: 0 of 2 input terms covered = 0.00%

Input Term	Covered	Reason for no coverage	Hint
opcode_reg[1]	N	'_1' not hit	Hit '_1'
opcode_reg[2]	N	'_1' not hit	Hit '_1'

Rows:	Hits	FEC Target	Non-masking condition(s)
ROW 1:	1	opcode_reg[1]_0	opcode_reg[2]
ROW 2:	***0***	opcode_reg[1]_1	opcode_reg[2]
ROW 3:	1	opcode_reg[2]_0	opcode_reg[1]
ROW 4:	***0***	opcode_reg[2]_1	opcode_reg[1]

-----Focused Expression View-----

Line 20 Item 1 (invalid_red_op | invalid_opcode)

Expression totals: 1 of 2 input terms covered = 50.00%

Input Term	Covered	Reason for no coverage	Hint
invalid_red_op	Y		
invalid_opcode	N	'_1' not hit	Hit '_1'

Rows:	Hits	FEC Target	Non-masking condition(s)
ROW 1:	1	invalid_red_op_0	~invalid_opcode
ROW 2:	1	invalid_red_op_1	~invalid_opcode
ROW 3:	1	invalid_opcode_0	~invalid_red_op
ROW 4:	***0***	invalid_opcode_1	~invalid_red_op

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Statements	48	48	0	100.00%

=====Statement Details=====

Statement Coverage for instance /question_3_tb/DUT --

Line	Item	Count	Source
-----	-----	-----	-----
File ALSU.v			
1			module ALSU(A, B, cin, serial_in, red_op_A, red_op_B, opcode, bypass_A, bypass_B, clk, rst, direction, leds, out);
2			parameter INPUT_PRIORITY = "A";
3			parameter FULL_ADDER = "ON";
4			input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
5			input [2:0] opcode;
6			input signed [2:0] A, B;
7			output reg [15:0] leds;
8			output reg signed [5:0] out;
9			
10			reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
11			reg signed [1:0] cin_reg;
12			reg [2:0] opcode_reg;
13			reg signed [2:0] A_reg, B_reg;
14			
15			wire invalid_red_op, invalid_opcode, invalid;
16			
17			//Invalid handling
18	1	973	assign invalid_red_op = (red_op_A_reg red_op_B_reg) & (opcode_reg[1] opcode_reg[2]);

18	1	973	assign invalid_red_op = (red_op_A_reg red_op_B_reg) & (opcode_reg[1] opcode_reg[2]);
19	1	840	assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
20	1	526	assign invalid = invalid_red_op invalid_opcode;
21			
22			//Registering input signals
23	1	1038	always @(posedge clk or posedge rst) begin
24			if(rst) begin
25	1	2	cin_reg <= 0;
26	1	2	red_op_B_reg <= 0;
27	1	2	red_op_A_reg <= 0;
28	1	2	bypass_B_reg <= 0;
29	1	2	bypass_A_reg <= 0;
30	1	2	direction_reg <= 0;
31	1	2	serial_in_reg <= 0;
32	1	2	opcode_reg <= 0;
33	1	2	A_reg <= 0;
34	1	2	B_reg <= 0;
35			end else begin
36	1	1036	cin_reg <= cin;
37	1	1036	red_op_B_reg <= red_op_B;
38	1	1036	red_op_A_reg <= red_op_A;
39	1	1036	bypass_B_reg <= bypass_B;

code_cvr_alu - Notepad

File Edit Format View Help

39	1	1036	bypass_B_reg <= bypass_B;
40	1	1036	bypass_A_reg <= bypass_A;
41	1	1036	direction_reg <= direction;
42	1	1036	serial_in_reg <= serial_in;
43	1	1036	opcode_reg <= opcode;
44	1	1036	A_reg <= A;
45	1	1036	B_reg <= B;
46			end
47			end
48			
49			//leds output blinking
50	1	1039	always @(posedge clk or posedge rst) begin
51			if(rst) begin
52	1	3	leds <= 0;
53			end else begin
54			if (invalid)
55	1	511	leds <= ~leds;
56			else
57	1	525	leds <= 0;
58			end
59			end
60			

60			
61			//ALU output processing
62	1	1038	always @(posedge clk or posedge rst) begin
63			if(rst) begin
64	1	2	out <= 0;
65			end
66			else begin
67			if (bypass_A_reg && bypass_B_reg)
68	1	226	out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
69			else if (bypass_A_reg)
70	1	262	out <= A_reg;
71			else if (bypass_B_reg)
72	1	249	out <= B_reg;
73			else if (invalid)
74	1	134	out <= 0;
75			else begin
76			case (opcode_reg) //edited design error
77			3'h0: begin
78			if (red_op_A_reg && red_op_B_reg)
79	1	7	out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
80			else if (red_op_A_reg)
81	1	8	out <= A_reg;

code_cvr_alu - Notepad				
File	Edit	Format	View	Help
81		1	8	out <= A_reg;
82				else if (red_op_B_reg)
83		1	9	out <= B_reg;
84				else
85		1	26	out <= A_reg B_reg;
86				end
87				3'h1: begin
88				if (red_op_A_reg && red_op_B_reg)
89		1	10	out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
90				else if (red_op_A_reg)
91		1	13	out <= ^A_reg;
92				else if (red_op_B_reg)
93		1	9	out <= ^B_reg;
94				else
95		1	12	out <= A_reg ^ B_reg;
96				end
97		1	15	3'h2: out <= (FULL_ADDER == "ON") ? (A_reg + B_reg + cin_reg) : (A_reg + B_reg); //edited design error
98		1	15	3'h3: out <= A_reg * B_reg;
99				3'h4: begin
100				if (direction_reg)
101		1	13	out <= {out[4:0], serial_in_reg};
102				else

code_cvr_alu - Notepad				
File	Edit	Format	View	Help
102				else
103		1	14	out <= {serial_in_reg, out[5:1]};
104				end
105				3'h5: begin
106				if (direction_reg)
107		1	10	out <= {out[4:0], out[5]};
108				else
109		1	4	out <= {out[0], out[5:1]};

Toggle Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	----	-----
Toggles	114	114	0	100.00%

code_cvr_alu - Notepad

File Edit Format View Help

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	-----	-----	-----
Toggles	114	114	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /question_3_tb/DUT --

Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
A[0-2]	1	1	100.00
A_reg[2-0]	1	1	100.00
B[0-2]	1	1	100.00
B_reg[2-0]	1	1	100.00
bypass_A	1	1	100.00
bypass_A_reg	1	1	100.00
bypass_B	1	1	100.00
bypass_B_reg	1	1	100.00
cin	1	1	100.00
cin_reg[0]	1	1	100.00
clk	1	1	100.00
direction	1	1	100.00
direction_reg	1	1	100.00
invalid	1	1	100.00
invalid_red_op	1	1	100.00
leds[15-0]	1	1	100.00
opcode[0-2]	1	1	100.00
opcode_reg[2-0]	1	1	100.00
out[5-0]	1	1	100.00
red_op_A	1	1	100.00
red_op_A_reg	1	1	100.00
red_op_B	1	1	100.00
red_op_B_reg	1	1	100.00
serial_in	1	1	100.00
serial_in_reg	1	1	100.00

Total Node Count = 57
Toggled Node Count = 57
Untoggled Node Count = 0

Toggle Coverage = 100.00% (114 of 114 bins)

=====

8. Functional Coverage report snippets (Questions 1, 2, and 3 only)

Covergroups

File Bookmarks Window Help

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_
/alsu_pkg/alsu_bxn		100.00%					
TYPE cvr_gp		100.00%	100	100.00...		✓	
CVP cvr_gp::A		100.00%	100	100.00...		✓	
bin A_data_0		29	1	100.00...		✓	
bin A_data_max		35	1	100.00...		✓	
bin A_data_min		39	1	100.00...		✓	
default bin A_data_default		155	-	-		✓	
CVP cvr_gp::B		100.00%	100	100.00...		✓	
bin B_data_0		34	1	100.00...		✓	
bin B_data_max		24	1	100.00...		✓	
bin B_data_min		37	1	100.00...		✓	
default bin B_data_default		163	-	-		✓	
CVP cvr_gp::opcode		100.00%	100	100.00...		✓	
illegal_bin bins_invalid		2	-	-		✓	
bin bins_shift[SHIFT_OP]		37	1	100.00...		✓	
bin bins_shift[ROT_OP]		38	1	100.00...		✓	
bin bins_arith[ADD_OP]		46	1	100.00...		✓	
bin bins_arith[MUL_OP]		49	1	100.00...		✓	
bin bins_bitwise[OR_OP]		41	1	100.00...		✓	
bin bins_bitwise[XOR_OP]		45	1	100.00...		✓	
INST \alsu_pkg::alsu_bxn::cvr_gp		100.00%	100	100.00...		✓	
CVP A		100.00%	100	100.00...		✓	
bin A_data_0		29	1	100.00...		✓	
bin A_data_max		35	1	100.00...		✓	
bin A_data_min		39	1	100.00...		✓	
default bin A_data_default		155	-	-		✓	
CVP B		100.00%	100	100.00...		✓	
bin B_data_0		34	1	100.00...		✓	
bin B_data_max		24	1	100.00...		✓	
bin B_data_min		37	1	100.00...		✓	
default bin B_data_default		163	-	-		✓	
CVP opcode		100.00%	100	100.00...		✓	
illegal_bin bins_invalid		2	-	-		✓	
bin bins_shift[SHIFT_OP]		37	1	100.00...		✓	
bin bins_shift[ROT_OP]		38	1	100.00...		✓	
bin bins_arith[ADD_OP]		46	1	100.00...		✓	
bin bins_arith[MUL_OP]		49	1	100.00...		✓	
bin bins_bitwise[OR_OP]		41	1	100.00...		✓	
bin bins_bitwise[XOR_OP]		45	1	100.00...		✓	

code_cvr_alu - Notepad

File Edit Format View Help

Covergroup Coverage:

Covergroups	1	na	na	100.00%
Coverpoints/Crosses	3	na	na	na
Covergroup Bins	12	12	0	100.00%

Covergroup	Metric	Goal	Bins	Status
TYPE /alsu_pkg/alsu_txn/cvr_gp	100.00%	100	-	Covered
covered/total bins:	12	12	-	
missing/total bins:	0	12	-	
% Hit:	100.00%	100	-	
Coverpoint A	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin A_data_0	29	1	-	Covered
bin A_data_max	35	1	-	Covered
bin A_data_min	39	1	-	Covered
default bin A_data_default	155		-	Occurred
Coverpoint B	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin B_data_0	34	1	-	Covered
bin B_data_max	24	1	-	Covered
bin B_data_min	37	1	-	Covered
default bin B_data_default	163		-	Occurred
Coverpoint opcode	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
illegal_bin bins_invalid	2		-	Occurred
bin bins_shift[SHIFT_OP]	37	1	-	Covered
bin bins_shift[ROT_OP]	38	1	-	Covered
bin bins_arith[ADD_OP]	46	1	-	Covered
bin bins_arith[MUL_OP]	49	1	-	Covered
bin bins_bitwise[OR_OP]	41	1	-	Covered
bin bins_bitwise[XOR_OP]	45	1	-	Covered
Covergroup instance \alsu_pkg::alsu_txn::cvr_gp	100.00%	100	-	Covered
covered/total bins:	12	12	-	
missing/total bins:	0	12	-	
% Hit:	100.00%	100	-	

code_cvr_alu - Notepad

File Edit Format View Help

% Hit:	100.00%	100	-	
Coverpoint A	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin A_data_0	29	1	-	Covered
bin A_data_max	35	1	-	Covered
bin A_data_min	39	1	-	Covered
default bin A_data_default	155		-	Occurred
Coverpoint B	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin B_data_0	34	1	-	Covered
bin B_data_max	24	1	-	Covered
bin B_data_min	37	1	-	Covered
default bin B_data_default	163		-	Occurred
Coverpoint opcode	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
illegal_bin bins_invalid	2		-	Occurred
bin bins_shift[SHIFT_OP]	37	1	-	Covered
bin bins_shift[ROT_OP]	38	1	-	Covered
bin bins_arith[ADD_OP]	46	1	-	Covered
bin bins_arith[MUL_OP]	49	1	-	Covered
bin bins_bitwise[OR_OP]	41	1	-	Covered
bin bins_bitwise[XOR_OP]	45	1	-	Covered

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	9	9	0	100.00%

..

Covergroup	Metric	Goal	Bins	Status
TYPE /alsu_pkg/alsu_txn/cvr_gp	100.00%	100	-	Covered
covered/total bins:	12	12	-	
missing/total bins:	0	12	-	
% Hit:	100.00%	100	-	
Coverpoint A	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin A_data_0	29	1	-	Covered
bin A_data_max	35	1	-	Covered
bin A_data_min	39	1	-	Covered
default bin A_data_default	155		-	Occurred
Coverpoint B	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin B_data_0	34	1	-	Covered
bin B_data_max	24	1	-	Covered
bin B_data_min	37	1	-	Covered
default bin B_data_default	163		-	Occurred
Coverpoint opcode	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
illegal_bin bins_invalid	2		-	Occurred
bin bins_shift[SHIFT_OP]	37	1	-	Covered
bin bins_shift[ROT_OP]	38	1	-	Covered
bin bins_arith[ADD_OP]	46	1	-	Covered
bin bins_arith[MUL_OP]	49	1	-	Covered
bin bins_bitwise[OR_OP]	41	1	-	Covered
bin bins_bitwise[XOR_OP]	45	1	-	Covered
Covergroup instance \alsu_pkg::alsu_txn::cvr_gp	100.00%	100	-	Covered
covered/total bins:	12	12	-	
missing/total bins:	0	12	-	
% Hit:	100.00%	100	-	
Coverpoint A	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	

code_cvr_alu - Notepad

File Edit Format View Help

bin bins_shift[SHIFT_OP]	37	1	-	Covered
bin bins_shift[ROT_OP]	38	1	-	Covered
bin bins_arith[ADD_OP]	46	1	-	Covered
bin bins_arith[MUL_OP]	49	1	-	Covered
bin bins_bitwise[OR_OP]	41	1	-	Covered
bin bins_bitwise[XOR_OP]	45	1	-	Covered
Covergroup instance \alsu_pkg::alsu_txn::cvr_gp	100.00%	100	-	Covered
covered/total bins:	12	12	-	
missing/total bins:	0	12	-	
% Hit:	100.00%	100	-	
Coverpoint A	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin A_data_0	29	1	-	Covered
bin A_data_max	35	1	-	Covered
bin A_data_min	39	1	-	Covered
default bin A_data_default	155		-	Occurred
Coverpoint B	100.00%	100	-	Covered
covered/total bins:	3	3	-	
missing/total bins:	0	3	-	
% Hit:	100.00%	100	-	
bin B_data_0	34	1	-	Covered
bin B_data_max	24	1	-	Covered
bin B_data_min	37	1	-	Covered
default bin B_data_default	163		-	Occurred
Coverpoint opcode	100.00%	100	-	Covered
covered/total bins:	6	6	-	
missing/total bins:	0	6	-	
% Hit:	100.00%	100	-	
illegal_bin bins_invalid	2		-	Occurred
bin bins_shift[SHIFT_OP]	37	1	-	Covered
bin bins_shift[ROT_OP]	38	1	-	Covered
bin bins_arith[ADD_OP]	46	1	-	Covered
bin bins_arith[MUL_OP]	49	1	-	Covered
bin bins_bitwise[OR_OP]	41	1	-	Covered
bin bins_bitwise[XOR_OP]	45	1	-	Covered

9. Clear and neat QuestaSim waveform snippets showing the functionality of the design

Q4

1. Testbench code

```
1  import mem_pkg::*;
2
3  module my_mem_tb;
4
5      // -----
6      // DUT Signals
7      // -----
8      logic clk;
9      logic write;
10     logic read;
11     logic [7:0] data_in;
12     logic [15:0] address;
13     logic [7:0] data_out;
14
15     // -----
16     // DUT Instantiation
17     // -----
18     my_mem dut (
19         .clk(clk),
20         .write(write),
21         .read(read),
22         .data_in(data_in),
23         .address(address),
24         .data_out(data_out)
25     );
26
27     // -----
28     // Clock Generation
29     // -----
30     initial begin
31         clk = 0;
32         forever #5 clk = ~clk; // 10ns clock period
33     end
34
```

```
35 // -----
36 // Initial Block
37 // -----
38 initial begin
39     // Initialize control signals
40     write = 0;
41     read = 0;
42     data_in = 0;
43     address = 0;
44
45     // 1. Generate stimulus
46     stimulus_gen();
47
48     // 2. Populate golden model
49     golden_model();
50
51     // 3. Write Phase
52     $display("Starting Write Phase...");
53     for (int i = 0; i < TESTS; i++) begin
54         @(negedge clk);
55         address = address_array[i];
56         data_in = data_to_write_array[i];
57         write = 1;
58         read = 0;
59
60         @(posedge clk); // Wait for write to complete
61     end
62     write = 0;
63
```

```

62     write = 0;
63
64     // 4. Read Phase and Self-Check
65     $display("Starting Read Phase...");
66     for (int i = 0; i < TESTS; i++) begin
67         @(negedge clk);
68         address = address_array[i];
69         write = 0;
70         read = 1;
71
72         @(posedge clk); // Wait for read to complete
73         check9Bits(address, {~^data_out, data_out}); // Includes parity check
74         data_read_queue.push_back({~^data_out, data_out});
75     end
76     read = 0;
77
78     // 5. Display Read Data from Queue
79     $display("Read Data Queue:");
80     while (data_read_queue.size() > 0) begin
81         static bit [8:0] temp = data_read_queue.pop_front();
82         $display("Data from queue: %0h", temp);
83     end
84
85     // 6. Final Report
86     $display("Simulation Completed.");
87     $display("Correct Count = %0d", correct_count);
88     $display("Error Count   = %0d", error_count);
89
90     $stop;
91 end
92
93 endmodule

```

2. Package code

```

1 package mem_pkg;
2
3 // Number of test operations
4 localparam int TESTS = 100;
5
6 // -----
7 // Data Structures
8 // -----
9 // Queues arrays for stimulus
10 bit [15:0] address_array[$]; // Random addresses
11 bit [7:0] data_to_write_array[$]; // Random data to be written
12
13 // Associative array for expected read values (includes parity)
14 bit [8:0] data_read_expect_assoc[int];
15
16 // Queue for actual data read from DUT
17 bit [8:0] data_read_queue[$];
18
19 // Error and correct counters
20 int error_count = 0;
21 int correct_count = 0;
22
23 // -----
24 // Task: Stimulus Generation
25 // -----
26 task automatic stimulus_gen();
27     $display("Generating Stimulus...");
28     address_array.delete();
29     data_to_write_array.delete();
30
31     for (int i = 0; i < TESTS; i++) begin
32         address_array.push_back($urandom_range(0, 65535));
33         data_to_write_array.push_back($urandom_range(0, 255));
34     end
35 endtask
36

```

```

37 // -----
38 // Task: Golden Model
39 // -----
40 task automatic golden_model();
41     $display("Populating Golden Model...");
42     data_read_expect_assoc.delete();
43
44     for (int i = 0; i < TESTS; i++) begin
45         bit parity = ~^data_to_write_array[i]; // Even parity
46         data_read_expect_assoc[address_array[i]] = {parity, data_to_write_array[i]};
47     end
48 endtask
49
50 // -----
51 // Task: Check 9-bit data
52 // -----
53 task automatic check9Bits(input bit [15:0] addr, input bit [8:0] actual_data);
54     if (actual_data !== data_read_expect_assoc[addr]) begin
55         $display("ERROR: Addr=%0h Expected=%0h Got=%0h", addr, data_read_expect_assoc[addr], actual_data);
56         error_count++;
57     end
58     else begin
59         $display("CORRECT: Addr=%0h Data=%0h", addr, actual_data);
60         correct_count++;
61     end
62 endtask
63
64 endpackage
65

```

3. Design code

```

1  module my_mem (
2      input logic      clk,
3      input logic      write,
4      input logic      read,
5      input logic [7:0] data_in,
6      input logic [15:0] address,
7      output logic [7:0] data_out
8  );
9
10     // Memory Declaration: 9-bit wide, 64K depth
11     logic [8:0] mem_array [0:65535];
12
13     always_ff @(posedge clk) begin
14         if (write) begin
15             // Even parity calculation
16             mem_array[address] <= {~^data_in, data_in};
17         end
18         else if (read) begin
19             data_out <= mem_array[address][7:0]; // Output only the 8-bit data
20         end
21     end
22
23 endmodule
24

```

4. Report any bugs detected in the design and fix them

5. Snippet to your verification plan document

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
MEM_1	RAM should write data correctly on positive edge of clk when write = 1.	Random addresses and random data values using address_array and data_to_write_array.	-	Checker ensures data stored in RAM matches the written value plus parity bit using check9Bits task.
MEM_2	RAM should read data correctly on positive edge of clk when read = 1.	Random addresses generated from address_array.	-	Checker compares data_out with expected data stored in data_read_expect_assoc.
MEM_3	Write operation has higher priority than read when both read and write are high at the same clock edge.	Directed test where both read and write are asserted together.	-	Checker ensures write happens and read is ignored when both are high.
MEM_4	Read and write cannot happen at the same time — only one operation allowed per clock cycle.	Directed test toggling read and write.	-	Assertion or checker verifies that both are not active simultaneously.
MEM_5	Even parity bit must be calculated correctly and stored as the MSB (9th bit) when data is written.	Random data writes, verify parity calculation $\sim^{\wedge}data_in$.	-	check9Bits task ensures parity matches even parity rule: parity_bit = $\sim^{\wedge}data_in$.
MEM_6	Data should persist at the given address after multiple operations (no data corruption).	Random writes followed by random reads to same addresses.	-	Checker ensures previously written data is retained unless overwritten.
MEM_7	Verify memory depth supports 64K addresses (16-bit address space).	Random addresses covering full range from 0 to 65535.	-	Checker ensures all addresses can be accessed and data matches expectations.
MEM_8	Output data queue should store and display all read values in order.	Capture read data into data_read_queue.	-	At end of simulation, pop and display values using pop_front to confirm order of reads is correct.
MEM_9	Verify error counting works correctly.	Introduce intentional mismatches (e.g., flip a bit in expected data).	-	Verify error_count increments on mismatch and correct_count increments on successful read.
MEM_10	Reset behavior: confirm no writes or reads occur when simulation starts with all signals stable.	Directed initial condition where read = 0 and write = 0.	-	Checker ensures no change in memory or outputs when idle.

6. Do file

```

1  vlib work
2  vlog my_mem.sv mem_pkg.sv my_mem_tb.sv +cover -covercells
3  vsim -voptargs=+acc work.my_mem_tb -cover
4  add wave *
5  coverage save mem.ucdb -onexit
6  run -all

```

7. Clear and neat QuestaSim waveform snippets showing the functionality of the design

